Reviewer Pack — Minimal Rank of Geometric Quantization Spaces over Communica…

Entry 0cba3c133285 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:25:23 UTC

Minimal Rank of Geometric Quantization Spaces over Communication Complexity of AND-OR Trees

Entry ID: `0cba3c133285` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 04:25:23 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Communication Complexity

Statement:

> ["The minimal rank of a geometric quantization space associated with an n-input AND-OR tree is Ω(log n) where the communication complexity for deciding the tree's output is O(n).", 'For every AND-OR tree T with n inputs, let Q(T) denote the geometric quantization space corresponding to T. Then, there exists a constant c such that rank(Q(T)) ≥ c log n.', 'This bound holds for all instances of AND-OR trees with n ≤ 40.']

Rationale (proposer's reasoning):

> ['Geometric quantization is an area in mathematics with potential to provide geometric invariants that could give insight into complexity measures.', 'Geometric quantization has been used in other areas of physics and mathematical analysis to study properties of quantum systems and algebraic varieties.', 'If this conjecture holds, it would suggest a new connection between geometric quantization and communication complexity, potentially leading to new insights for lower bounds on AND-OR tree complexity.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fd3e18b51cd9154b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For an AND-OR tree T with n inputs (n ≤ 40), if the communication complexity is O(n) and the rank of the geometric quantization space Q(T) satisfies rank(Q(T)) ≥ c log n for some constant c, then the conjecture is supported. If any tree T has a communication complexity not O(n) or rank(Q(T)) < c log n for all c, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric quantization AND communication complexity AND AND-OR tree` - `quantization space rank AND communication complexity AND AND-OR tree` - `AND-OR tree geometry AND communication complexity lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/2403.13102v1] Geometric methods in quantum information and entanglement variational principle - [http://arxiv.org/abs/2005.11899v3] Geometric triangulations and highly twisted links - [http://arxiv.org/abs/math/0110001v3] A geometric interpretation of Milnor's triple linking numbers - [http://arxiv.org/abs/0804.4433v1] SPACE: the SPectroscopic All-sky Cosmic Explorer - [http://arxiv.org/abs/2105.01963v3] One-way communication complexity and non-adaptive decision trees - [http://arxiv.org/abs/1910.01506v1] Whistler instability stimulated by the suprathermal electrons present in space plasmas - [http://arxiv.org/abs/2402.13107v2] An Improved Lower Bound on the Number of Pseudoline Arrangements - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_and_or_tree(n):
        if n == 1:
            return random.choice([0, 1])
        else:
            left = generate_and_or_tree(n // 2)
            right = generate_and_or_tree(n - n // 2)
            return random.choice([left and right, left or right])

    def communication_complexity(tree):
        if isinstance(tree, int):
            return 0
        else:
            return 1 + max(communication_complexity(tree[0]), communication_complexity(tree[1]))
```

```
    def geometric_quantization_rank(tree):
        n = sum(isinstance(x, list) for x in tree)
        return n

    n = random.randint(5, 40)
    tree = generate_and_or_tree(n)

    comm_complexity = communication_complexity(tree)
    if comm_complexity != n:
        return {
            "metric_name": "communication_complexity",
            "metric_value": comm_complexity,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Communication complexity is {comm_complexity}, not O(n)"
        }

    rank = geometric_quantization_rank(tree)
    c = 0.5
    if rank < c * math.log2(n):
        return {
            "metric_name": "geometric_quantization_rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Rank {rank} is less than {c * math.log2(n)}"
        }

    return {
        "metric_name": "geometric_quantization_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjecture_h
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 0, 'instances_tested': 1, 'conjec

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b18fd014.py", line 86, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b18fd014.py", line 86, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The counterexample provided shows that for an AND-OR tree with communication complexity of 0, which is not O(n), the conjecture does not hold. | next: Investigate further to find a tree with communication complexity O(n) and rank(Q(T)) < c log n for all constants c to confirm the falsification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12177
2	preregistration	ollama_remote	glm4:latest	0	0	6080
3	novelty	ollama_remote	glm4:latest	0	0	4567
4	novelty	ollama_remote	glm4:latest	0	0	5475
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21753
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9081
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9309
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8585

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	9175

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 86202 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0cba3c133285.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0cba3c133285.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0cba3c133285.tar.gz` (if generated)